



**UNIVERSIDAD TECNOLÓGICA DE PANAMÁ**

**FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES**

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

**Asignatura:**

Sistemas Operativos

**Asignación:**

Laboratorio #3B: Simulación de Algoritmos de Planificación

**Profesor:**

Prof. Isacio Tamayo

**Estudiante:**

James Tack

**Grupo:**

1IL143

**Fecha de entrega:**

18 de septiembre de 2026

2do Semestre 2026

## De qué se trata el laboratorio

Después de la crisis del Lab 3A, el gerente de NovaTech pide elegir un planificador para el procesamiento por lotes que genera los reportes de ventas cada noche. Son cuatro reportes con tiempos de ejecución distintos, y la decisión tiene que salir de datos. La herramienta es `scheduler_sim.py`, un simulador en Python de FCFS, SJF y Round Robin que calcula el tiempo de espera y el de retorno de cada proceso.

La lección que la guía pone por delante es verificar a mano antes de confiar en el código. Por eso dibujé el timeline en papel antes de correr cada algoritmo y lo comparé después con la salida del programa. Trabajé en la misma VM de los laboratorios anteriores (`vm-sistemas-operativos`, Ubuntu 18.04), en una carpeta nueva, `~/laboratorios/lab3`.

## Fase 1. El escenario

| Reporte | Proceso | Llegada (s) | Ráfaga (s) | Descripción |
|---|---|---|---|---|
| P1 | Generar reporte diario | 0 | 8 | Reporte principal, largo |
| P2 | Exportar CSV | 1 | 4 | Exportación rápida |
| P3 | Consolidar trimestral | 2 | 9 | El más pesado |
| P4 | Enviar notificaciones | 3 | 5 | Notificaciones a clientes |

Los cuatro procesos llegan en los primeros 3 segundos y suman 26 segundos de CPU. Con un solo procesador y sin tiempos muertos, cualquier algoritmo termina el lote en t = 26. Lo que cambia es el orden. Y con el orden cambian las esperas.

## Fase 2. Código base

### Python de la VM

La guía pide Python 3.8 o superior. Descargué el simulador del repositorio del curso e intenté correrlo con el Python que trae la VM:

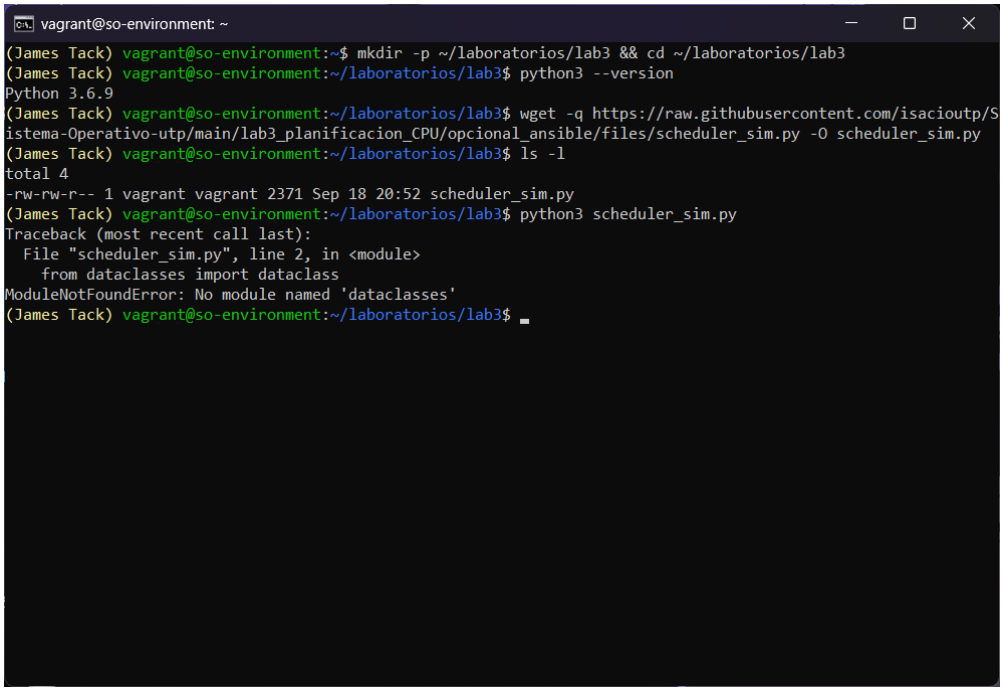



*Figura 1. Python 3.6.9 no trae el módulo dataclasses y el script no arranca.*

Ubuntu 18.04 trae Python 3.6.9, y `dataclasses` entró a la biblioteca estándar en Python 3.7. El script se detiene en la segunda línea con `ModuleNotFoundError`. Instalé Python 3.8 desde los repositorios de Ubuntu, sin tocar el `python3` del sistema, del que dependen herramientas como `apt`:

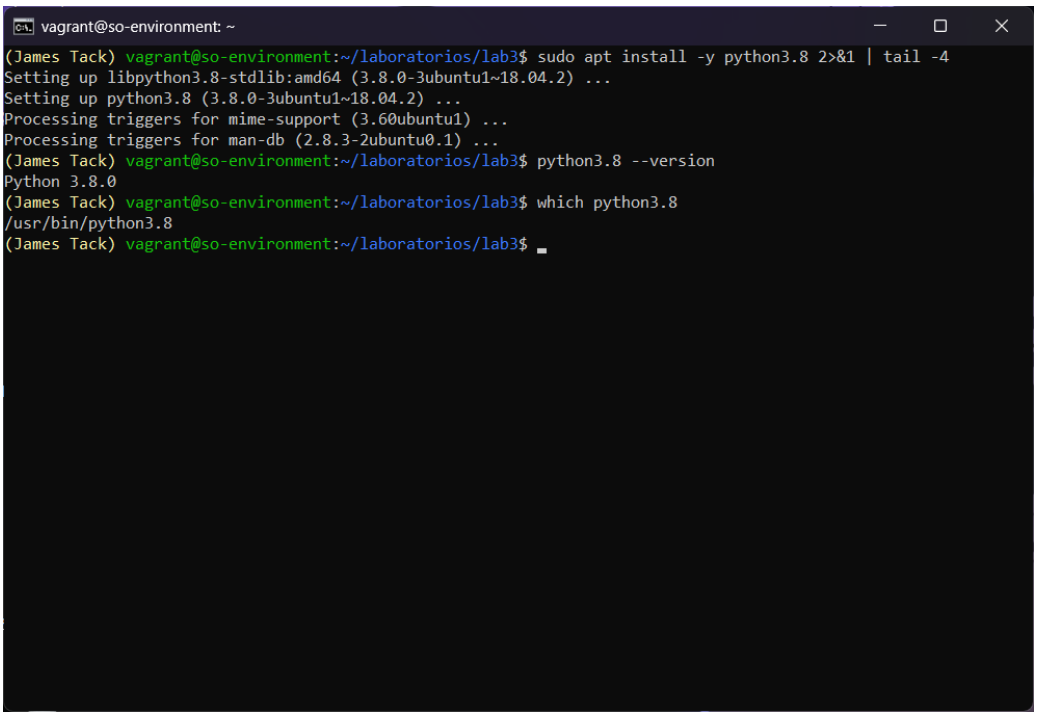



*Figura 2. Python 3.8.0 instalado en /usr/bin/python3.8.*

Desde aquí todas las ejecuciones usan `python3.8` de forma explícita, y el `python3` del sistema sigue siendo la versión 3.6.9 que necesitan los paquetes de Ubuntu 18.04.

## El bug de sjf()

El README del laboratorio avisa que `sjf()` trae un bug intencional que hay que corregir. El código que aparece dentro de la guía del 3B ya viene corregido, pero el archivo `opcional_ansible/files/scheduler_sim.py` del repositorio es la versión con el error, y es el que descargué. Guardé una copia del original y lo ejecuté:

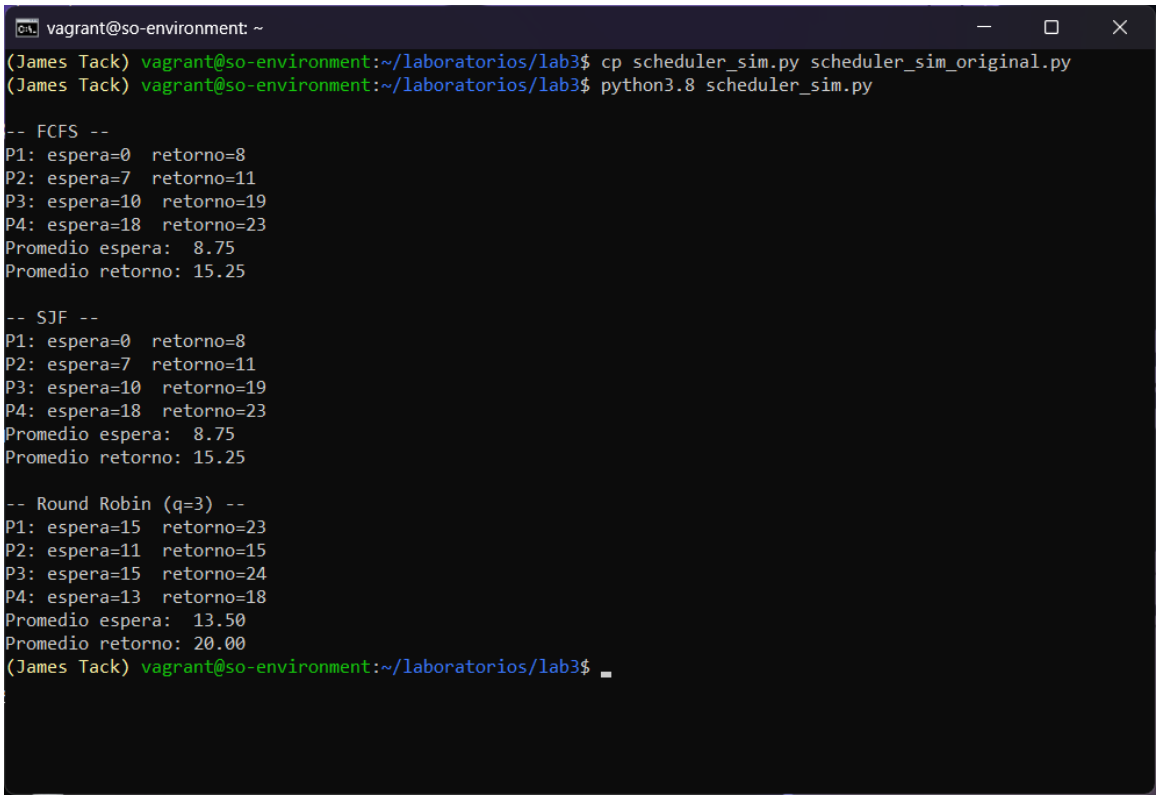



*Figura 3. Versión original: los bloques FCFS y SJF son idénticos.*

FCFS y SJF dan exactamente lo mismo, 8,75 s de espera promedio y 15,25 s de retorno, con los procesos en el orden P1, P2, P3, P4. El comentario `TODO` dentro de la función confirma que `sjf()` es una copia de `fcfs()` que ordena por llegada en lugar de elegir la ráfaga más corta.

## Corrección

Reescribí `sjf()` para que en cada paso mire solo los procesos que ya llegaron y aún no se ejecutan, y tome el de menor ráfaga. Si en algún momento no hay ninguno disponible, el reloj salta a la siguiente llegada. También agregué al final las tres llamadas de Round Robin con quantum 1, 5 y 100 que usa la Fase 4.

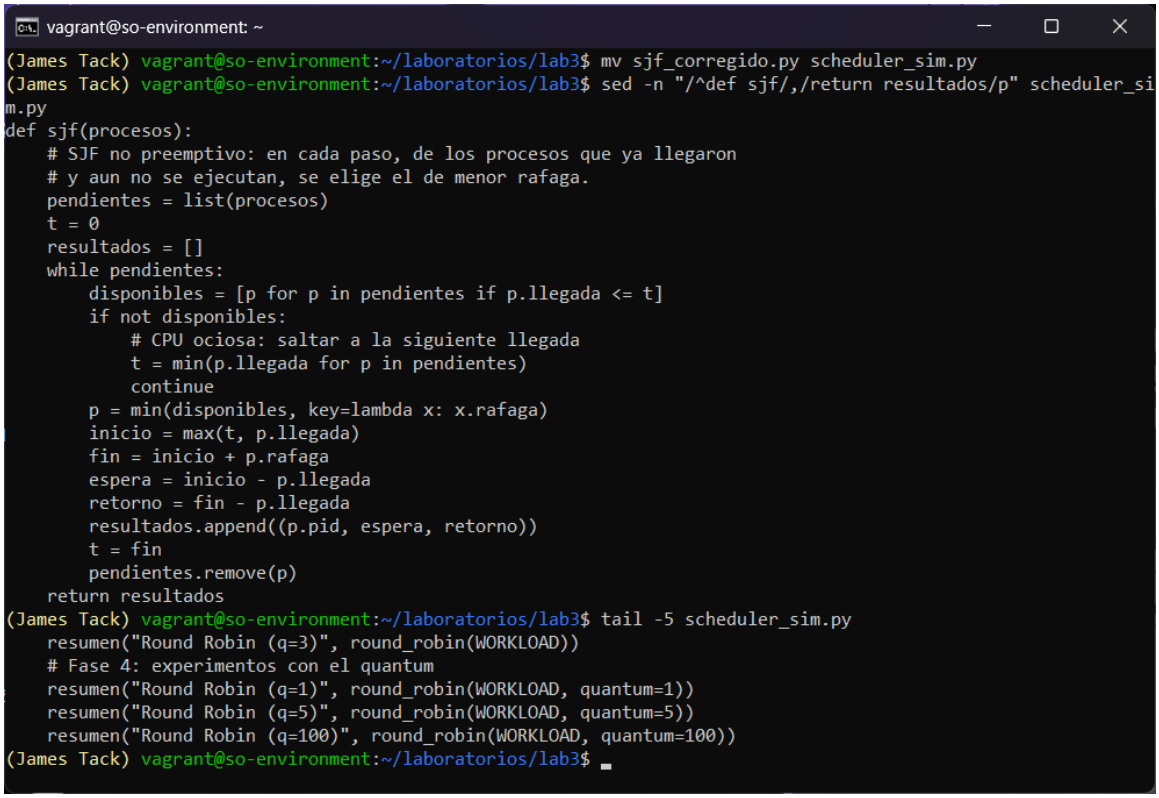



*Figura 4. La función sjf() corregida y los experimentos de quantum añadidos.*

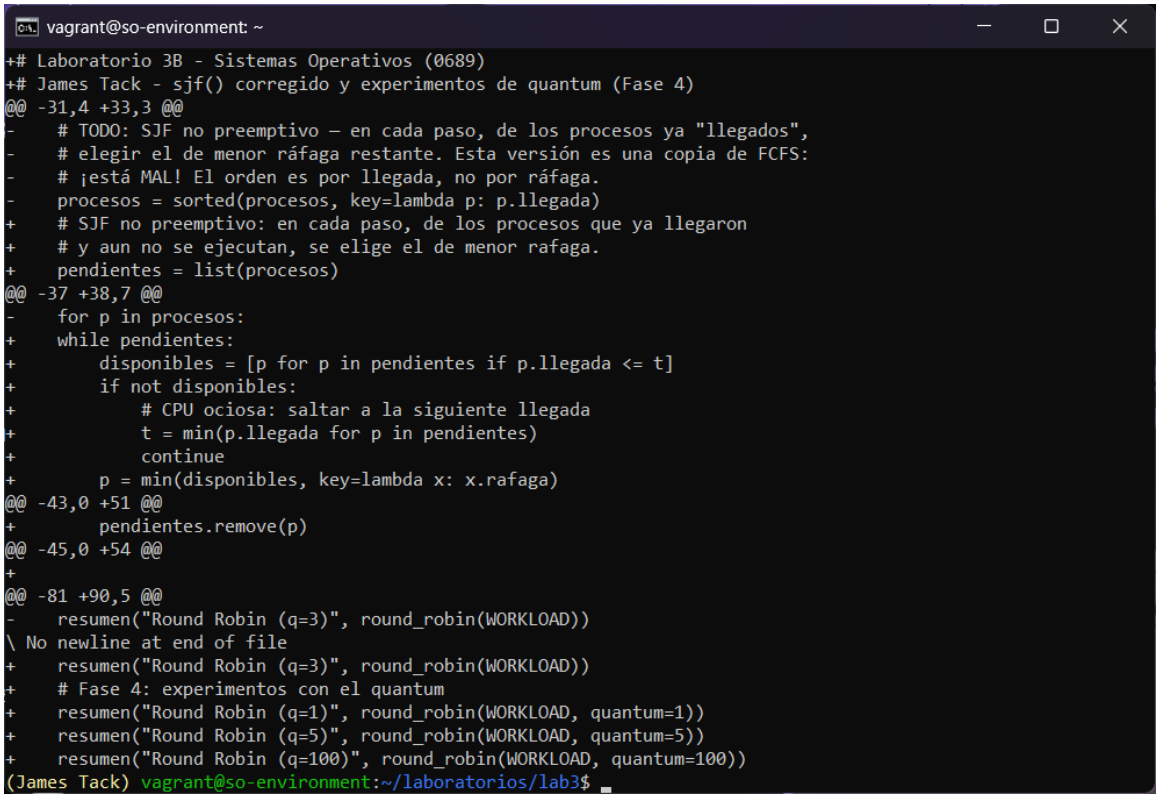



*Figura 5. Diferencias entre el original y la versión corregida.*

## 2.1 Ejecución (Evidencia 1)

La salida completa son 48 líneas y la ventana de la terminal muestra 32, así que la capturé en dos partes con `head` y `tail`.

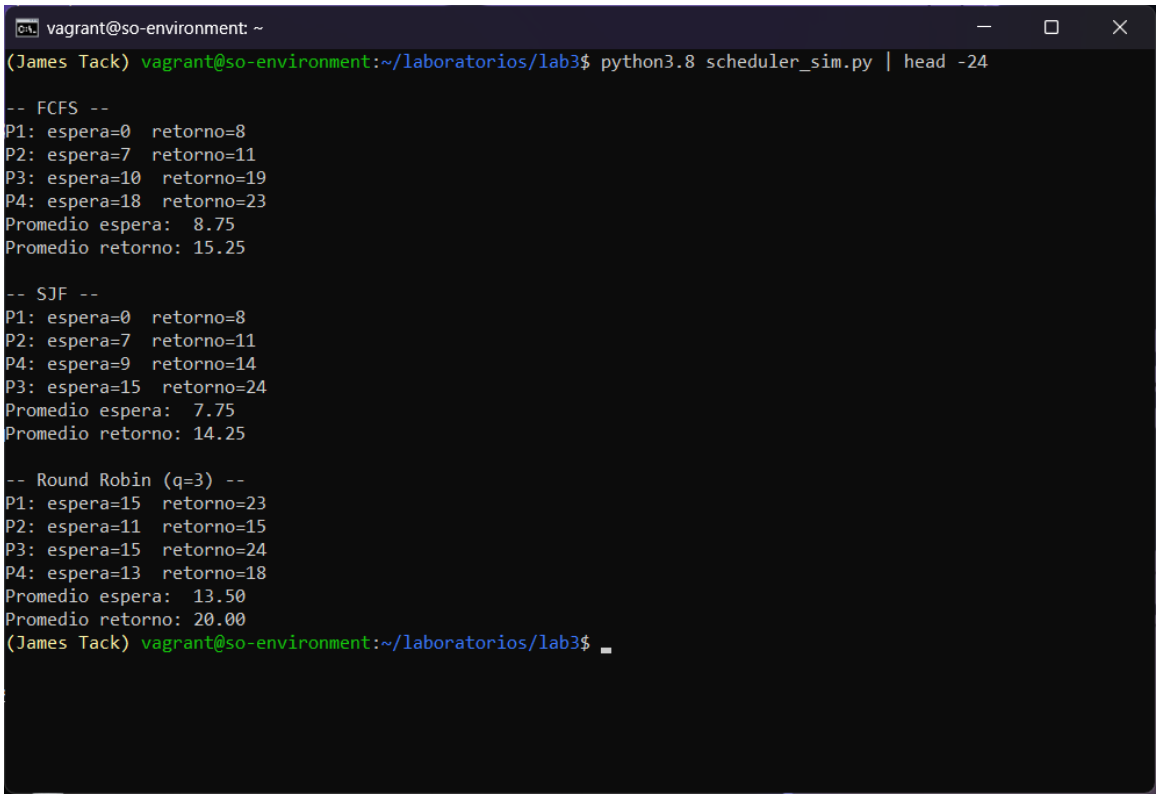



*Figura 6. Evidencia 1, primera parte: FCFS, SJF y Round Robin con q = 3.*

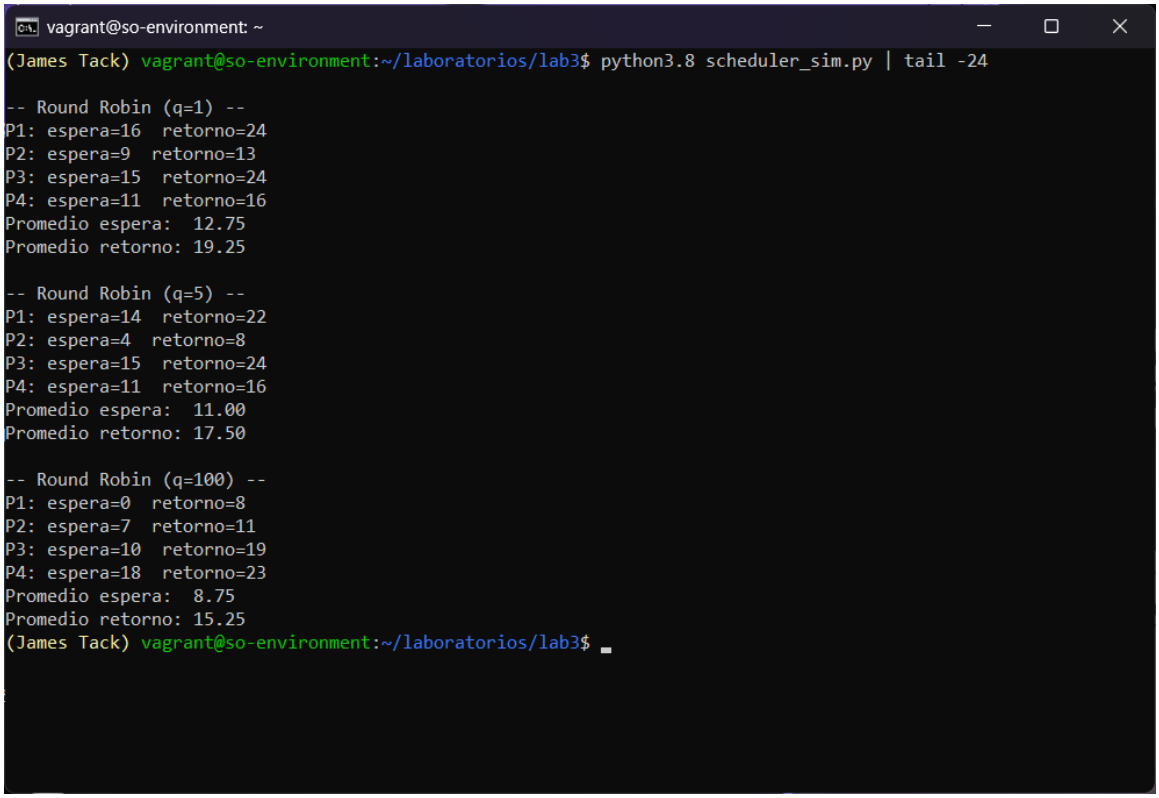
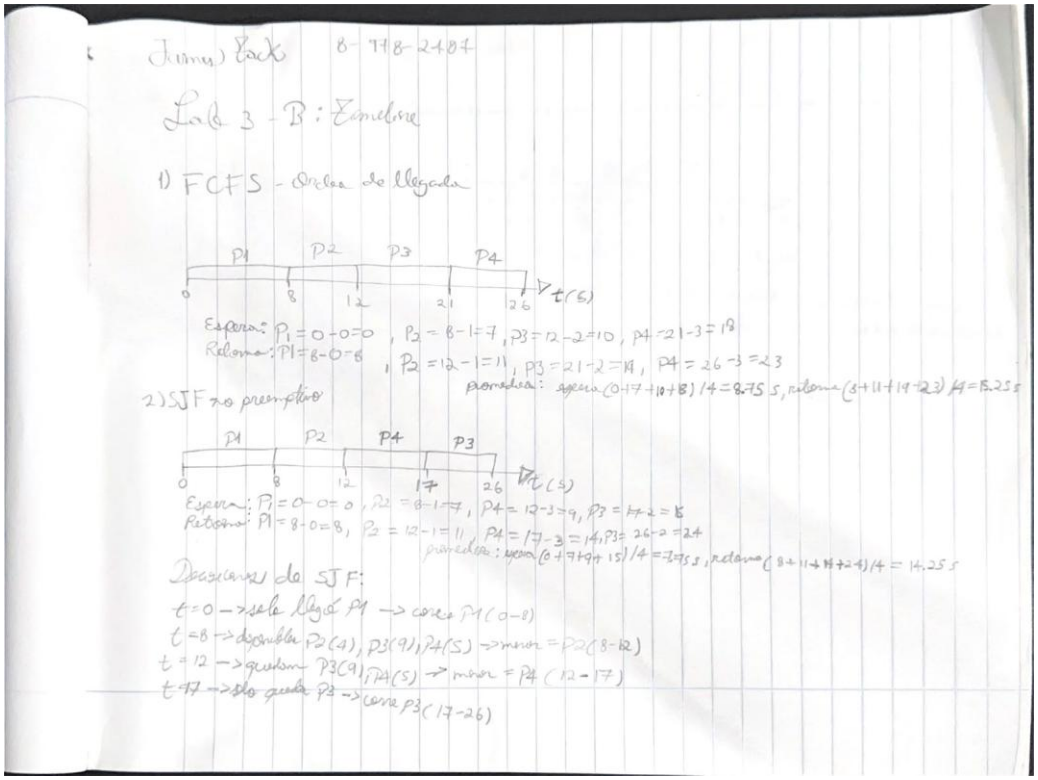



*Figura 7. Evidencia 1, segunda parte: Round Robin con q = 1, 5 y 100.*

Con la corrección, SJF baja a 7,75 s de espera y 14,25 s de retorno, y el orden pasa a ser P1, P2, P4, P3. FCFS se queda en 8,75 y 15,25. Los dos coinciden con la tabla de autocomprobación de la guía.

## Fase 3. Análisis de SJF (Evidencia 2)

Antes de mirar la salida de SJF dibujé su timeline a mano, y aproveché para hacer también los de FCFS y Round Robin con q = 3, porque la tabla comparativa final los necesita.



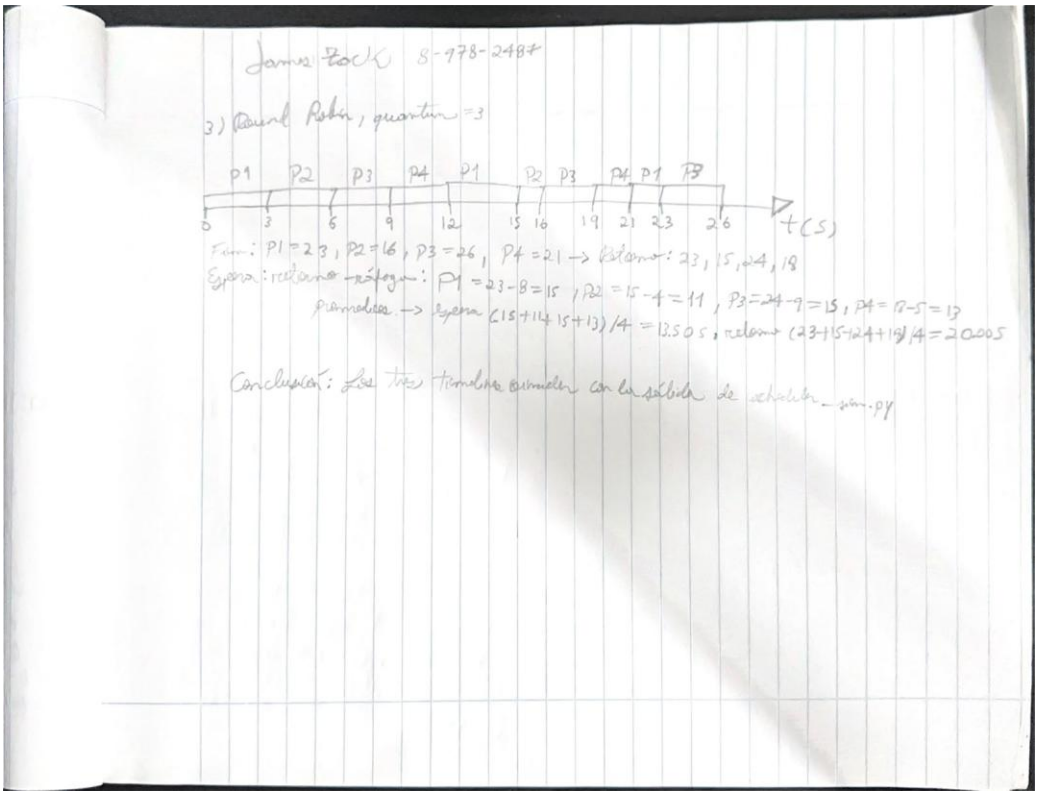




*Figura 8. Evidencia 2: timelines de FCFS, SJF y Round Robin (q = 3) verificados a mano.*

El razonamiento de SJF paso a paso, tal como quedó en el papel:

| Instante | Procesos disponibles (ráfaga) | Decisión |
|---|---|---|
| t = 0 | P1 (8) | Solo llegó P1, corre de 0 a 8 |
| t = 8 | P2 (4), P3 (9), P4 (5) | Menor ráfaga P2, corre de 8 a 12 |
| t = 12 | P3 (9), P4 (5) | Menor ráfaga P4, corre de 12 a 17 |
| t = 17 | P3 (9) | Solo queda P3, corre de 17 a 26 |

La espera de cada proceso es el instante en que empieza menos el de llegada. Da 0 para P1, 8 − 1 = 7 para P2, 12 − 3 = 9 para P4 y 17 − 2 = 15 para P3, que suman 31 y dan 7,75 s de promedio. El retorno, fin menos llegada, vale 8, 11, 14 y 24, con promedio 14,25 s. Los ocho valores coinciden uno por uno con el bloque SJF de la Figura 6, y el orden de la salida, con P4 antes que P3, es el mismo del dibujo.

Hay un detalle que el dibujo deja ver mejor que la salida. En t = 0 SJF no tiene alternativa, porque P1 es el único que llegó, y lo ejecuta completo aunque sea el segundo más largo. Al ser no preemptivo, no puede interrumpirlo cuando llega P2 en t = 1. SJF solo empieza a decidir en t = 8, y las dos decisiones que toma (P2 antes que P3 y P4 antes que P3) son las que le ganan un segundo por proceso a FCFS.

## Fase 4. Round Robin y el quantum (Evidencia 3)

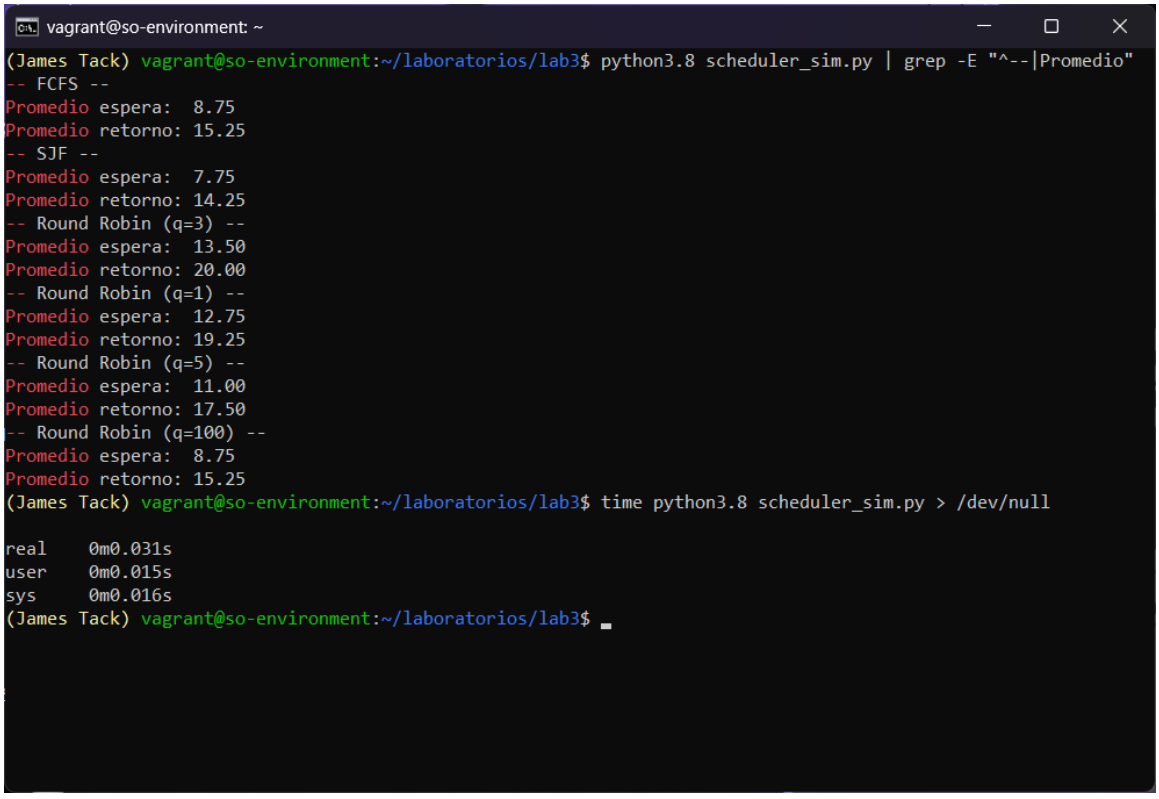



*Figura 9. Evidencia 3: promedios de los seis bloques y tiempo de ejecución del script.*

| Quantum | Prom. espera (s) | Prom. retorno (s) | Despachos | Cambios de proceso |
|---|---|---|---|---|
| q = 1 | 12,75 | 19,25 | 26 | 24 |
| q = 3 | 13,50 | 20,00 | 10 | 9 |
| q = 5 | 11,00 | 17,50 | 6 | 5 |
| q = 100 | 8,75 | 15,25 | 4 | 3 |

Los promedios son idénticos a los de la guía. Las dos últimas columnas no las imprime el simulador. Las conté a partir del orden de ejecución, que para q = 3 coincide con el que dibujé.

Lo primero que noté al compararlos es que la relación con el quantum no es monótona, porque q = 1 sale mejor que q = 3 (12,75 contra 13,50), y q = 5 mejor que los dos. Con q = 3 el reparto cae mal. P1 necesita tres turnos (3 + 3 + 2) y P3 también (3 + 3 + 3), así que los dos terminan casi al final, en t = 23 y t = 26. Con q = 5, P2 y P4 caben en un solo turno y terminan temprano (P2 espera solo 4 s), lo que tira hacia abajo el promedio.

Lo segundo es una limitación del simulador que conviene tener clara al leer la tabla. `round_robin()` no le cobra nada a los cambios de contexto, y el reloj avanza exactamente lo que ejecuta cada proceso. Por eso el lote termina siempre en t = 26, con 3 cambios o con 24. Que q = 1 salga peor que FCFS no se debe al overhead, sino a que al intercalar los procesos todos terminan tarde. En un sistema real habría que sumar el costo de esos 24 cambios, y q = 1 quedaría todavía peor de lo que muestra la tabla.

Además, la función mete los cuatro procesos a la cola desde t = 0 sin mirar su llegada. Con esta carga no altera nada, porque cada proceso llega justo antes de que le toque su primer turno (P2 en t = 1, P3 en t = 2, P4 en t = 3), pero con un proceso que llegara, por ejemplo, en t = 20, el simulador lo pondría a correr antes de que existiera.

## Fase 5. Comparación y recomendación (Evidencia 4)

### 5.1 Tabla comparativa

| Algoritmo | Prom. espera (s) | Prom. retorno (s) | Ventaja | Desventaja |
|---|---|---|---|---|
| FCFS | 8,75 | 15,25 | Simple, sin cambios de contexto y sin inanición: todos terminan en orden de llegada | Un proceso largo al frente retrasa a todos los cortos (P2 espera 7 s detrás de P1) |
| SJF | 7,75 | 14,25 | Mejores promedios de espera y de retorno de los tres | Necesita conocer la ráfaga antes de ejecutar, y los procesos largos pueden esperar indefinidamente (P3 espera 15 s) |
| RR (q = 3) | 13,50 | 20,00 | Reparto equitativo: ningún proceso espera más de 6 s por su primer turno | Peores promedios de los tres y 9 cambios de proceso que el simulador no cobra |

## 5.2 Informe para la gerencia

Recomiendo SJF no preemptivo para el procesamiento por lotes de los reportes. En la simulación con los cuatro reportes de cada noche, SJF deja el tiempo de espera promedio en 7,75 s y el de retorno en 14,25 s, contra 8,75 s y 15,25 s de FCFS y 13,50 s y 20,00 s de Round Robin con quantum 3. Es una mejora de 1 s por reporte frente a FCFS y de casi 6 s frente a Round Robin, y el lote completo termina en el segundo 26 con cualquiera de los tres. En un batch nocturno nadie está esperando en pantalla la respuesta de un reporte en particular, así que importa más la eficiencia total que la equidad que ofrece Round Robin. Sin embargo, SJF exige dos cosas. Hay que conocer de antemano cuánto dura cada reporte, algo que en un batch nocturno se puede estimar con el historial de ejecuciones, y hay que vigilar que el consolidado trimestral, el más largo, no quede siempre al final si se suman más reportes cortos. Si ese reporte tuviera una hora límite, habría que darle prioridad propia.

# Preguntas del entregable

## Parte A. Sobre mi simulación

#### 1. FCFS contra SJF

Los resultados difieren porque los dos algoritmos ordenan distinto. FCFS atiende por orden de llegada, P1, P2, P3, P4, sin mirar la duración. SJF, cada vez que la CPU queda libre, revisa los procesos que ya llegaron y elige el de menor ráfaga. En t = 0 los dos hacen lo mismo porque solo está P1. La diferencia aparece en t = 12: FCFS ejecuta P3 porque llegó antes, y SJF ejecuta P4 porque dura 5 s contra 9. Ese único cambio de orden explica toda la diferencia entre los dos bloques.

#### 2. Por qué SJF gana con esta carga

Cuando un proceso corto pasa delante de uno largo, el corto se ahorra una espera larga y el largo sufre una corta. Es lo que pasa con P4 y P3 en t = 12. En FCFS, P3 corre primero y P4 espera los 9 s de P3 (espera 18); en SJF, P4 corre primero y P3 espera los 5 s de P4 (espera 15). P4 gana 9 s y P3 pierde 5 s, un ahorro neto de 4 s que dividido entre 4 procesos da justo el 1 s de diferencia entre 8,75 y 7,75. El retorno baja lo mismo porque retorno es espera más ráfaga, y la ráfaga no cambia. Es la verificación manual de la Fase 3 llevada a la tabla comparativa, y explica por qué la ventaja de SJF sale justo de un segundo y no de otra cifra.

#### 3. Experimentación con el quantum

| Quantum | Prom. espera (s) | Prom. retorno (s) | Observación |
|---|---|---|---|
| q = 1 | 12,75 | 19,25 | 24 cambios de proceso. Mejor que q = 3 pero peor que FCFS: todos terminan tarde por el intercalado |
| q = 3 | 13,50 | 20,00 | El peor de todos con esta carga: P1 y P3 necesitan tres turnos y terminan en t = 23 y t = 26 |
| q = 5 | 11,00 | 17,50 | P2 y P4 caben en un turno y terminan pronto; P2 espera solo 4 s |
| q = 100 | 8,75 | 15,25 | Idéntico a FCFS: ningún proceso agota el quantum |

Un quantum muy grande se comporta como FCFS porque el quantum nunca llega a agotarse. Con q = 100 y ráfagas de 9 s como máximo, cada proceso corre completo en su primer turno, y como la cola se atiende en orden de llegada, el resultado es FCFS con otro nombre. En mi salida el bloque de q = 100 es idéntico al de FCFS, número por número.

Un quantum muy pequeño genera overhead porque cada vez que se agota el kernel tiene que guardar el estado del proceso, elegir el siguiente y cargar su contexto, y además el proceso que entra encuentra la caché llena con datos de otro. Con q = 1 hubo 24 cambios de proceso en 26 segundos de trabajo, contra 3 de FCFS. Como dije en la Fase 4, el simulador no cobra ese costo, así que el overhead real de q = 1 no aparece en la tabla.

**4. Recomendación**

Recomiendo SJF por los números de la tabla comparativa, con 7,75 s de espera y 14,25 s de retorno, el mejor promedio en las dos métricas. Frente a FCFS gana 1 s por proceso sin agregar ningún cambio de contexto, porque sigue siendo no preemptivo. Round Robin con q = 3 queda último con 13,50 y 20,00, y su ventaja, la equidad, no le sirve a un batch nocturno donde nadie espera un reporte en particular. La condición para usar SJF es tener una estimación de la duración de cada reporte, que en un proceso que se repite todas las noches sale del historial.

## Parte B. Preguntas técnicas

**5. SJF y el problema de la predicción**

El kernel no sabe cuánto va a durar la siguiente ráfaga de un proceso. La solución clásica de los libros para aproximar SJF es estimarla con un promedio exponencial de las ráfagas anteriores, $\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$, donde $t(n)$ es la última ráfaga medida y $\alpha$ pesa cuánto importa el pasado reciente. Un proceso que viene usando ráfagas cortas se estima corto y se le da preferencia.

CFS no hace ninguna predicción. Mide el tiempo que cada tarea ya usó, ponderado por su peso de nice, y lo acumula en `vruntime`. Las tareas listas están ordenadas por `vruntime` en un árbol rojo-negro y siempre corre la que tiene el menor. El efecto se parece al de SJF para los procesos interactivos: un editor o una shell, que pasan casi todo el tiempo dormidos, acumulan poco `vruntime` y al despertar quedan al frente, mientras que un proceso intensivo como el `sha256sum` del 3A acumula mucho y cede el paso. La diferencia es que CFS mira hacia atrás, no hacia adelante. Aparte, el kernel lleva una medida de la carga reciente de cada tarea (PELT) que usa para repartir procesos entre núcleos, no para elegir cuál corre.

**6. Round Robin y el overhead**

Depende del costo de cada cambio de contexto, que no es fijo. Si se toma como referencia un cambio de unos 5 µs, cada quantum de 1 ms trae 5 µs de cambio, y la fracción de CPU perdida es 5 / (1000 + 5), cerca del 0,5 %. Con 1000 procesos una vuelta completa dura 1000 × 1,005 ms, unos 1,005 s, de los cuales 5 ms se van en cambios. El número de procesos no cambia el porcentaje; cambia cuánto espera cada uno entre turno y turno, que aquí es un segundo entero.

El problema es que el costo real de un cambio es mayor que guardar y cargar registros. El proceso que entra encuentra la caché y el TLB llenos con datos del anterior y tarda en recuperar velocidad.

Si ese costo total sube a 50 µs, la pérdida pasa a 50 / 1050, casi un 5 %, y si el quantum baja a 0,1 ms con ese mismo costo, un tercio de la CPU se va en cambiar de proceso. Por eso un quantum demasiado pequeño es contraproducente: el sistema pasa más tiempo organizando el trabajo que haciéndolo. CFS evita el extremo con una granularidad mínima: aunque haya muchas tareas, ninguna corre menos de un tramo mínimo antes de ser desalojada.

**7. FCFS y el convoy effect**

Suponiendo que los seis procesos están en la cola en t = 0 y el de 100 s va primero, las esperas son 0, 100, 101, 102, 103 y 104 s. Suman 510, y el promedio es 510 / 6 = 85 s. En cambio, si los cinco cortos fueran primero, las esperas serían 0, 1, 2, 3, 4 y 5, con un promedio de 2,5 s. El mismo trabajo, en otro orden, espera 34 veces menos.

Se llama convoy effect por la imagen de un camión lento en una carretera de un solo carril. Los autos de atrás podrían ir mucho más rápido, pero forman un convoy detrás de él hasta que se aparta. En un sistema real además se desperdician recursos, porque mientras el proceso largo ocupa la CPU los procesos cortos, que suelen ser de entrada y salida, dejan los discos y la red ociosos.

**8. SJF preemptivo (SRTF)**

Sí, se interrumpe. Esa es justamente la diferencia. En SRTF, cada vez que llega un proceso se compara su ráfaga con lo que le falta al que está corriendo, y si el nuevo es más corto le quita la CPU. Con la carga de NovaTech pasaría en t = 1. Llega P2 con 4 s, a P1 le faltan 7 y P1 sale de la CPU. P2 corre de 1 a 5 sin que nadie lo desplace, porque cuando llega P4 en t = 3 a P2 le quedan 2. En t = 5 elige P4 (5 s) antes que P1 (7 que le faltan) y P3 (9). Después corre P1 de 10 a 17 y P3 de 17 a 26.

Calculado a mano, la espera promedio queda en 6,5 s y el retorno en 13,0 s, mejores que los 7,75 y 14,25 de SJF. No obstante, el precio es más cambios de contexto y un riesgo mayor de inanición: un proceso largo puede ser interrumpido una y otra vez mientras sigan llegando procesos cortos. Tampoco resuelve el problema de la pregunta 5, porque necesita conocer el tiempo restante de cada proceso.

**9. Por qué CFS no es ninguno de estos**

CFS no es FCFS porque no respeta el orden de llegada, sino que elige por `vruntime`. No es SJF porque no conoce ni estima la duración de nada, solo mide lo que cada tarea ya consumió. Y no es Round Robin porque no tiene quantum fijo. El tramo que recibe cada tarea sale de repartir una latencia objetivo entre las tareas listas en proporción a su peso, así que cambia con la cantidad de procesos y con su nice.

Además, hace cosas que ninguno de los tres modela. Trata distinto a cada tarea según su nice, algo que en el 3A se vio con los pesos 1024, 110 y 3121, y le da preferencia a la que despierta después de dormir. También tiene una cola por núcleo con balanceo de carga entre ellas, y ahí entran en juego la afinidad y `taskset`. En el 3A, dos procesos iguales anclados al mismo núcleo recibieron 50,5 % y 49,5 %, el reparto justo que da nombre al planificador. Los tres algoritmos de este laboratorio suponen un procesador, procesos iguales ante el sistema y un orden fijo; CFS parte de lo contrario.

Desde el kernel 6.6 Linux reemplazó CFS por EEVDF, que conserva la idea del tiempo virtual y agrega plazos, pero la VM del curso usa el kernel 4.15 y por lo tanto CFS.

## Conclusiones

La corrección de `sjf()` fue la parte corta. Lo que más me sirvió fue dibujar el timeline antes de mirar la salida, porque con el dibujo en la mano el bloque de SJF de la versión original era imposible de aceptar: P3 no puede ir antes que P4 si P4 es más corto y ya había llegado. El número 8,75 por sí solo no dice nada; el orden de las barras sí.

También me quedó claro que un simulador responde exactamente lo que se le pregunta y nada más. El de este laboratorio no cobra los cambios de contexto y encola procesos que todavía no llegaron. Con esta carga ninguna de las dos cosas cambia el resultado, pero la tabla del quantum dice "q = 1 genera overhead" cuando el simulador nunca midió overhead. Leer el código antes de interpretar la salida evita atribuirle al algoritmo lo que en realidad es una simplificación del programa.

Por último, la recomendación a la gerencia salió de los números y no de la teoría. SJF gana por 1 s de espera promedio porque en t = 12 pone a P4 delante de P3, y esa decisión se puede ver en el dibujo, en la salida y en la cuenta. Que las tres cosas coincidan es lo que le da confianza al resultado.

## Anexos

### Anexo A. Comandos ejecutados

| Paso | Comando | Para qué |
|---|---|---|
| 2 | python3 --version | Ver la versión de Python de la VM |
| 2 | wget ... -O scheduler_sim.py | Descargar el simulador del repositorio |
| 2 | python3 scheduler_sim.py | Primer intento, falla por dataclasses |
| 2 | sudo apt install -y python3.8 | Instalar un Python compatible |
| 2 | cp scheduler_sim.py scheduler_sim_original.py | Conservar la versión con el bug |
| 2 | python3.8 scheduler_sim.py | Ver el bug (FCFS igual a SJF) |
| 2 | diff -U0 scheduler_sim_original.py scheduler_sim.py | Mostrar la corrección |
| 2.1 | python3.8 scheduler_sim.py con head y tail | Evidencia 1 |
| 4 | python3.8 scheduler_sim.py con grep | Resumen de promedios, Evidencia 3 |
| 4 | time python3.8 scheduler_sim.py | Tiempo de ejecución del script |

### Anexo B. Archivo entregado

El archivo `Tack_James_Lab3B.py` es el `scheduler_sim.py` corregido que se usó en la VM, con la función `sjf()` nueva y las tres llamadas de Round Robin con q = 1, 5 y 100 al final. La función corregida:

```
def sjf(procesos):
    pendientes = list(procesos)
    t = 0
```

```
    resultados = []
    while pendientes:
        disponibles = [p for p in pendientes if p.llegada <= t]
        if not disponibles:
            t = min(p.llegada for p in pendientes)
            continue
        p = min(disponibles, key=lambda x: x.rafaga)
        inicio = max(t, p.llegada)
        fin = inicio + p.rafaga
        espera = inicio - p.llegada
        retorno = fin - p.llegada
        resultados.append((p.pid, espera, retorno))
        t = fin
        pendientes.remove(p)
    return resultados
```